目　　录

实验一　基本运算器实验.....错误!未定义书签。

一、实验目的.....错误!未定义书签。

二、实验设备.....错误!未定义书签。

三、实验原理.....错误!未定义书签。

四、实验步骤.....6

五、实验结果.....7

实验二　静态随机存储器实验.....9

一、实验目的.....9

二、实验设备.....9

三、实验原理.....9

四、实验步骤.....12

五、实验结果.....错误!未定义书签。5

实验三　系统总线与总线接口.....16

一、实验目的.....错误!未定义书签。

二、实验设备.....错误!未定义书签。

三、实验原理.....16

四、实验步骤.....19

五、实验结果.....24

实验五　简单模型机设计实验.....27

一、实验目的.....27

二、实验设备.....27

三、实验原理.....27

四、实验步骤.....33

五、实验结果.....36

实验一　基本运算器实验

计算机的一个最主要的功能就是处理各种算术和逻辑运算，这个功能要由 CPU 中的运算器来完成，运算器也称作算术逻辑部件 ALU。通过基本的运算器实验，了解运算器的基本结构。

一、　实验目的

(1) 了解运算器的组成结构。

(2) 掌握运算器的工作原理。

二、　实验设备

PC 机一台，TD-CMA 实验系统一套。

三、　实验原理

本实验的原理如图 1-1 所示。

运算器内部含有三个独立运算**部件，分**别为算术、逻辑和移位运算部件，要处理的数据存于暂存器 A 和暂存器 B，三个部件同时接受来自 A 和 B 的数据（有些处理器体系结构把移位运算器放于算术和逻辑运算部件之前，如 ARM），各部件对操作数进行何种运算由控制信号 S3…S0 和 CN 来决定，任何时候，多路选择开关只选择三部件中一个部件的结果作为 ALU 的输出。如果是影响进位的运算，还将置进位标志 FC，在运算结果输出前，置 ALU 零标志。ALU 中所有模块集成在一片 CPLD 中。

逻辑运算部件由逻辑门构成，较为简单，而后面又有专门的算术运算部件设计实验，在此对这两个部件不再赘述。移位运算采用的是桶形移位器，一般采用交叉开关矩阵来实现，交叉开关的原理如图 1-2 所示。图中显示的是一个 4×4 的矩阵（系统中是一个 8×8 的矩阵)。每一个输入都通过开关与一个输出相连，把沿对角线的开关导通，就可实现移位功能，即:

(1) 对于逻辑左移或逻辑右移功能，将一条对角线的开关导通，这将所有的输入位与所使用的输出分别相连,而没有同任何输入相连的则输出连接 0。

(2) 对于循环右移功能，右移对角线同互补的左移对角线一起激活。例如，在 4 位矩阵中使用‘右 1’和‘左 3’对角线来实现右循环 1 位。

(3) 对于未连接的输出位，移位时使用符号扩展或是 0 填充，具体由相应的指令控制。使用另外的逻辑进行移位总量译码和符号判别。

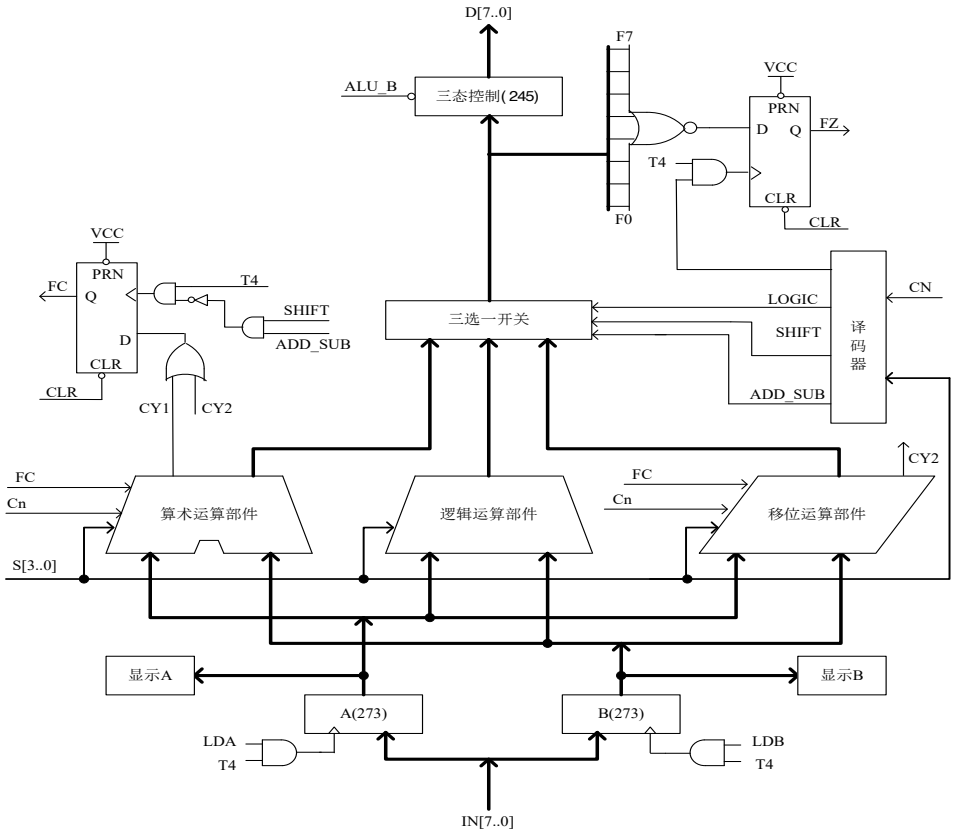


图 1-1　运算器原理图

运算器部件由一片 CPLD 实现。ALU 的输入和输出通过三态门 74LS245 连到 CPU 内总线上，另外还有指示灯标明进位标志 FC 和零标志 FZ。请注意：实验箱上凡是印标注有马蹄形标记‘ ⊔ ’，表示这两根排针之间是连通的。图中除 T4 和 CLR，其余信号均来自于 ALU 单元的排线座，实验箱中所有单元的 T1、T2、T3、T4 都连接至控制总线单元的 T1、T2、T3、T4，CLR 都连接至 CON 单元的 CLR 按钮。T4 由时序单元的 TS4 提供，其余控制信号均由 CON 单元的二进制数据开关模拟给出。控制信

号中除 T4 为脉冲信号外，其余均为电平信号，其中 ALU_B 为低有效，其余为高有效。

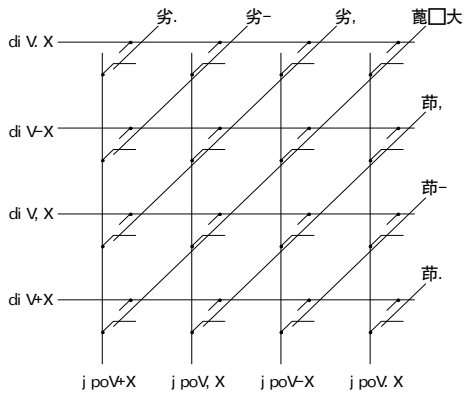


图 1-2　交叉开关桶形移位器原理图

暂存器 A 和暂存器 B 的数据能在 LED 灯上实时显示，原理如图 1-3 所示(以 A0 为例,其它相同)。进位标志 FC、零标志 FZ 和数据总线 D7…D0 的显示原理也是如此。

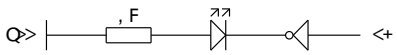


图 1-3　A0 显示原理图

ALU 和外围电路的连接如图 1-4 所示，图中的小方框代表排针座。

运算器的逻辑功能表如表 1-1 所示,其中 S3 S2 S1 S0 CN 为控制信号，FC 为进位标志，FZ 为运算器零标志，表中功能栏内的 FC、FZ 表示当前运算会影响到该标志。

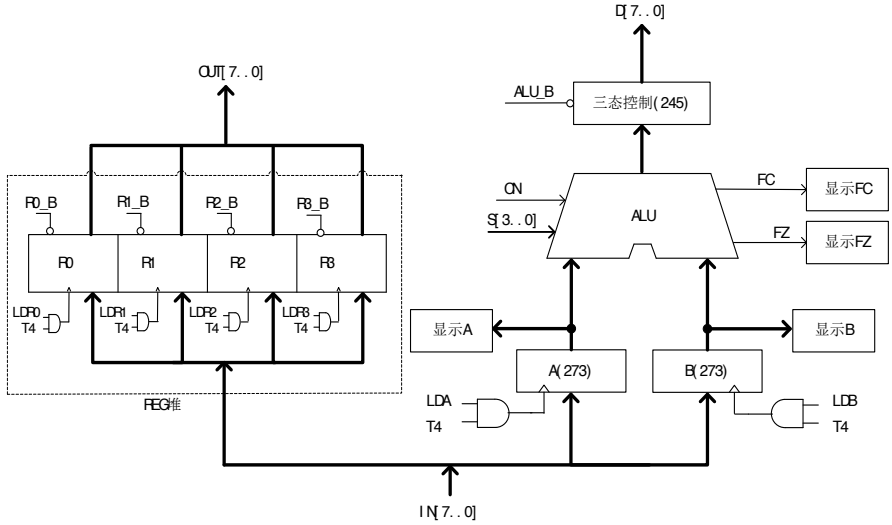


图 1-4　ALU 和外围电路连接原理图

表 1-1　运算器逻辑功能表

运算类型	S3 S2 S1 S0	CN	功　能
逻辑运算	0000	X	F=A（直通）
	0001	X	F=B（直通）
	0010	X	F=AB　　(FZ)
	0011	X	F=A+B　　(FZ)
	0100	X	F=/A　　(FZ)
移位运算	0101	X	F=A 不带进位循环右移 B（取低 3 位）位　　(FZ)
	0110	0	F=A 逻辑右移一位　　(FZ)
		1	F=A 带进位循环右移一位　　(FC，FZ)
	0111	0	F=A 逻辑左移一位　　(FZ)
		1	F=A 带进位循环左移一位　　(FC，FZ)
算术运算	1000	X	置 FC=CN　　(FC)
	1001	X	F=A 加 B　　(FC，FZ)
	1010	X	F=A 加 B 加 FC　　(FC，FZ)
	1011	X	F=A 减 B　　(FC，FZ)
	1100	X	F=A 减 1　　(FC，FZ)
	1101	X	F=A 加 1　　(FC，FZ)
	1110	X	（保留）
	1111	X	（保留）

*表中“X”为任意态，下同

四、 实验步骤

(1) 按图 1-5 连接实验电路，并检查无误。图中将用户需要连接的信号用圆圈标明（其它实验相同）。

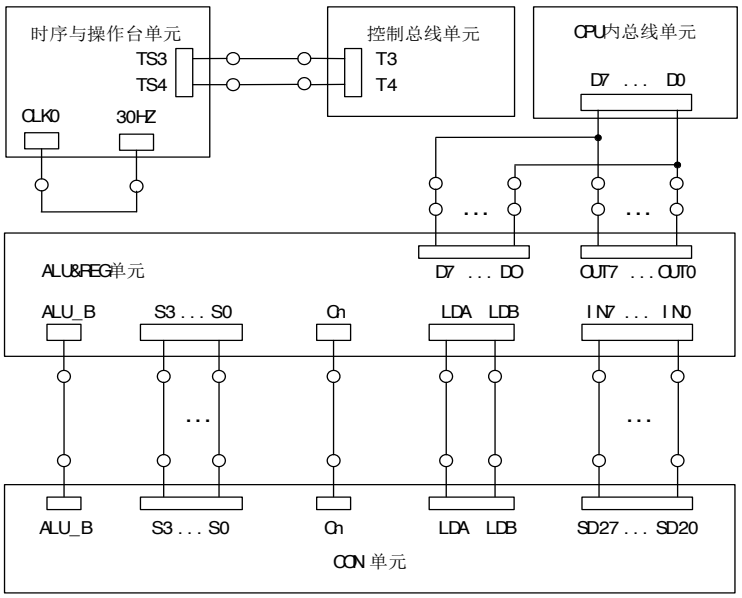


图 1-5 实验接线图

(2) 将时序与操作台单元的开关 KK2 置为‘单步’档,开关 KK1、KK3 置为‘运行’档。

(3) 打开电源开关，如果听到有‘嘀’报警声，说明有总线竞争现象，应立即关闭电源，重新检查接线，直到错误排除。然后按动 CON 单元的 CLR 按钮，将运算器的 A、B 和 FC、FZ 清零。

(4) 用输入开关向暂存器 A 置数。

① 拨动 CON 单元的 SD27…SD20 数据开关，形成二进制数 01100101（或

其它数值)，数据显示亮为‘1’，灭为‘0’。

② 置 LDA=1，LDB=0，ALU-B=1，按动时序单元的 ST 按钮，产生一个 T4 上沿，则将二进制数 01100101 置入暂存器 A 中，暂存器 A 的值通过 ALU 单元的 A7…A0 八位 LED 灯显示。

(5) 用输入开关向暂存器 B 置数。

① 拨动 CON 单元的 SD27…SD20 数据开关，形成二进制数 10100111（或其它数值)。

② 置 LDA=0，LDB=1，按动时序单元的 ST 按钮，产生一个 T4 上沿，则将二进制数 10100111

置入暂存器 B 中，暂存器 B 的值通过 ALU 单元的 B7…B0 八位 LED 灯显示。

(6) 改变运算器的功能设置，观察运算器的输出。置 ALU_B=0、LDA=0、LDB=0，然后按表 1-1 置 S3、S2、S1、S0 和 Cn 的数值，并观察数据总线 LED 显示灯显示的结果。如置 S3、S2、S1、S0 为 0010，运算器作逻辑与运算，置 S3、S2、S1、S0 为 1001，运算器作加法运算。

五、实验结果

表 1-2　运算结果表

运算类型	A	B	S3 S2 S1 S0	CN	结果
逻辑运算	65	A7	0　0　0　0	X	F=(65)　FC=(0)　FZ=(0)
	65	A7	0　0　0　1	X	F=(A7)　FC=(0)　FZ=(0)

			0 0 1 0	X	F=(25) FC=(0) FZ=(0)
			0 0 1 1	X	F=(E7) FC=(0) FZ=(0)
			0 1 0 0	X	F=(9A) FC=(0) FZ=(0)
移位运算			0 1 0 1	X	F=(CA) FC=(0) FZ=(0)
			0 1 1 0	0	F=(32) FC=(0) FZ=(0)
				1	F=(B2) FC=(1) FZ=(0)
			0 1 1 1	0	F=(CA) FC=(1) FZ=(0)
				1	F=(CA) FC=(0) FZ=(0)
算术运算			1 0 0 0	X	F=(65) FC=(0) FZ=(0)
			1 0 0 1	X	F=(0C) FC=(1) FZ=(0)
			1 0 1 0（FC=0）	X	F=(0C) FC=(0) FZ=(0)
			1 0 1 0（FC=1）	X	F=(0D) FC=(1) FZ=(0)
			1 0 1 1	X	F=(BE) FC=(1) FZ=(0)
			1 1 0 0	X	F=(64) FC=(0) FZ=(0)
			1 1 0 1	X	F=(66) FC=(0) FZ=(0)

实验二　静态随机存储器实验

存储器是计算机各种信息存储与交换的中心。在程序执行过程中，所要执行的指令是从存储器中获取，运算器所需要的操作数是通过程序中的访问存储器指令从存储器中得到，运算结果在程序执行完之前又必须全部写到存储器中，各种输入输出设备也直接与存储器交换数据。把程序和数据存储在存储器中，是冯·诺依曼型计算机的基本特征，也是计算机能够自动、连续快速工作的基础。

一、 实验目的

掌握静态随机存储器 RAM 工作特性及数据的读写方法。

二、 实验设备

PC 机一台，TD-CMA 实验系统一套。

三、 实验原理

实验所用的静态存储器由一片 6116（2K×8bit）构成（位于 MEM 单元），如图 2-1 所示。6116 有三个控制线：CS（片选线）、OE（读线）、WE（写线），其功能如表 2-1 所示，当片选有效（CS=0）时，OE=0 时进行读操作，WE=0 时进行写操作，本实验将 CS 常接地。

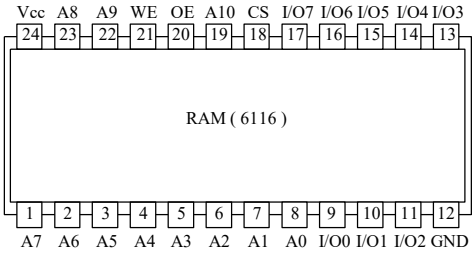


图 2-1　SRAM 6116 引脚图

由于存储器（MEM）最终是要挂接到 CPU 上，所以其还需要一个读写控制逻辑，使得 CPU 能控制 MEM 的读写，实验中的读写控制逻辑如图 2-2 所示，由于 T3 的参与，可以保证 MEM 的写脉宽与 T3 一致，T3 由时序单元的 TS3 给出。IOM 用来选择是对 I/O 还是对 MEM 进行读写操作，RD=1 时为读，WR=1 时为写。

表 2-1　SRAM 6116 功能表

>N	R@	J @	谨□
,	|	|	蓖□昀
+	,	+	□
+	+	,	画
+	+	+	画

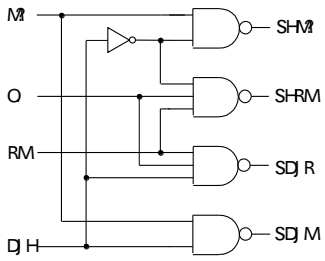


图 2-2　读写控制逻辑

实验原理图如图 2-3 所示，存储器数据线接至数据总线，数据总线上

接有 8 个 LED 灯显示 D7…D0 的内容。地址线接至地址总线，地址总线上接有 8 个 LED 灯显示 A7…A0 的内容，地址由地址锁存器（74LS273，位于 PC&AR 单元）给出。数据开关（位于 IN 单元）经一个三态门（74LS245）连至数据总线，分时给出地址和数据。地址寄存器为 8 位，接入 6116 的地址 A7…A0，6116 的高三位地址 A10…A8 接地，所以其实际容量为 256 字节。

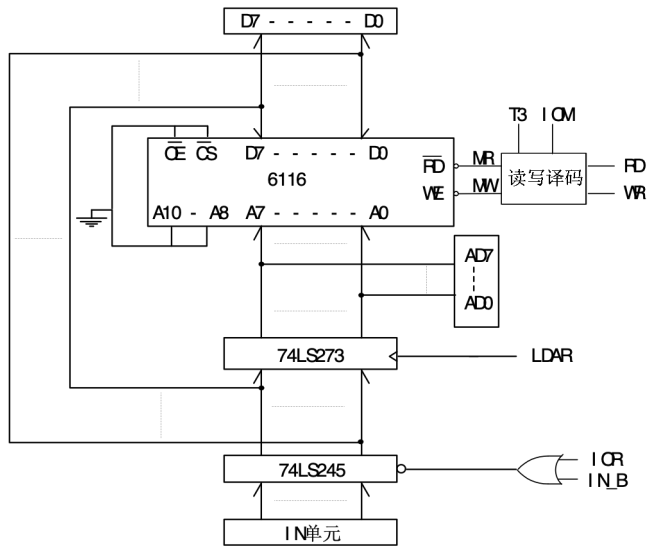


图 2-3　存储器实验原理图

实验箱中所有单元的时序都连接至时序与操作台单元，CLR 都连接至 CON 单元的 CLR 按钮。实验时 T3 由时序单元给出，其余信号由 CON 单元的二进制开关模拟给出，其中 IOM 应为低（即 MEM 操作），RD、WR 高有效，MR 和 MW 低有效，LDAR 高有效。

四、实验步骤

(1) 关闭实验系统电源，按图 2-4 连接实验电路，并检查无误，图中将用户需要连接的信号用圆圈标明。

(2) 将时序与操作台单元的开关 KK1、KK3 置为运行档、开关 KK2 置为‘单步’档。

(3) 将 CON 单元的 IOR 开关置为 1（使 IN 单元无输出），打开电源开关，如果听到有‘嘀’报警声，说明有总线竞争现象，应立即关闭电源，重新检查接线，直到错误排除。

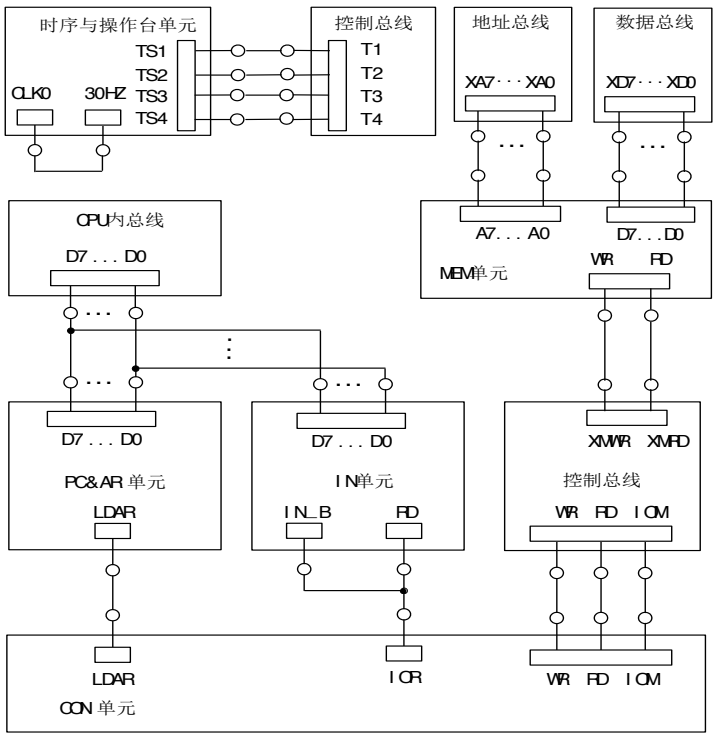


图 2-4　实验接线图

(4) 给存储器的 00H、01H、02H、03H、04H 地址单元中分别写入数据 11H、12H、13H、14H、15H。由前面的存储器实验原理图（图 2-3）可以看出，由于数据和地址由同一个数据开关给出，因此数据和地址要分时写入，先写地址，具体操作步骤为：先关掉存储器的读写（WR=0，RD=0），数据开关输出地址（IOR=0），然后打开地址寄存器门控信号（LDAR=1），按动 ST 产生 T3 脉冲，即将地址打入到 AR 中。再写数据，具体操作步骤为：先关掉存储器的读写（WR=0，RD=0）和地址寄存器门控信号（LDAR=0），数据开关输出要写入的数据，打开输入三态门（IOR=0），然后使存储器处于写状态（WR=1，RD=0，IOM=0），按动 ST 产生 T3 脉冲，即将数据打入到存储器中。写存储器的流程如图 2-5 所示（以向 00 地址单元写入 11H 为例）：

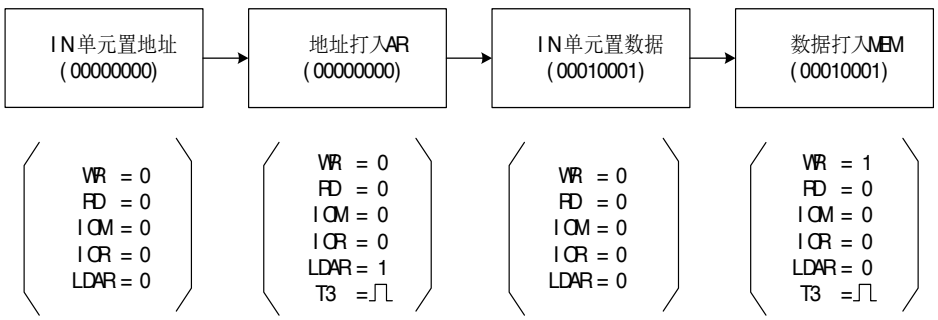


图 2-5　写存储器流程图

(5) 依次读出第 00、01、02、03、04 号单元中的内容，观察上述各单元中的内容是否与前面写入的一致。同写操作类似，也要先给出地址，然

后进行读，地址的给出和前面一样，而在进行读操作时，应先关闭 IN 单元的输出（IOR=1），然后使存储器处于读状态（WR=0，RD=1，IOM=0），此时数据总线上的数即为从存储器当前地址中读出的数据内容。读存储器的流程如图 2-6 所示（以从 00 地址单元读出 11H 为例）：

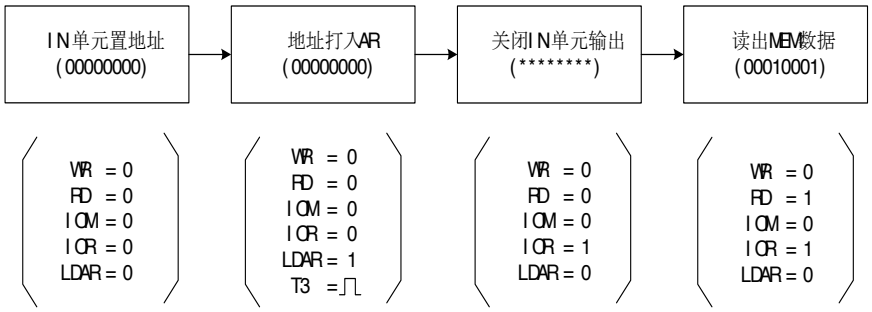


图 2-6　读存储器流程图

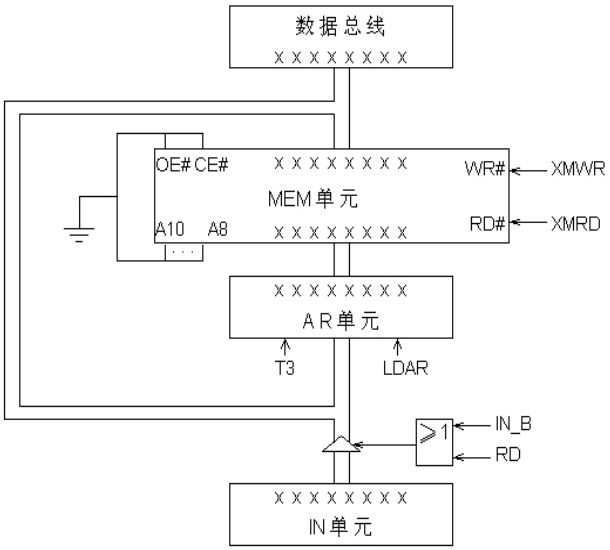


图 2-7　数据通路图

五、实验结果

地址	写入数据	读出数据
00H	11H	11H
01H	12H	12H
02H	13H	13H
03H	14H	14H
04H	15H	15H
11H	10H	10H
22H	20H	20H

实验三　系统总线与总线接口

总线是计算机中连接各个功能部件的纽带，是计算机各部件之间进行信息传输的公共通路。总线不只是一组简单的信号传输线，它还是一组协议。分时与共享是总线的两大特征。所谓共享，在总线上可以挂接多个部件，它们都可以使用这一信息通路来和其他部件传送信息。所谓分时，同一总线在同一时刻，只能有一个部件占领总线发送信息，其他部件要发送信息得在该部件发送完释放总线后才能申请使用。总线结构是决定计算机性能、功能、可扩展性和标准化程度的重要因素。

一、实验目的

1．理解总线的概念及其特性。

2．掌握控制总线的功能和应用。

二、实验设备

PC 机一台，TD-CMA 实验系统一套。

三、实验原理

由于存储器和输入、输出设备最终是要挂接到外部总线上，所以需要外部总线提供数据信号、地址信号以及控制信号。在该实验平台中，外部总线分为数据总线、地址总线、和控制总线，分别为外设提供上述信号。外部总线和 CPU 内总线之间通过三态门连接，同时实现了内外总线的分离

和对于数据流向的控制。地址总线可以为外部设备提供地址信号和片选信号。由地址总线的高位进行译码，系统的 I/O 地址译码原理见图 3-1（在地址总线单元)。由于使用 A6、A7 进行译码， I/O 地址空间被分为四个区，如表 3-1 所示：

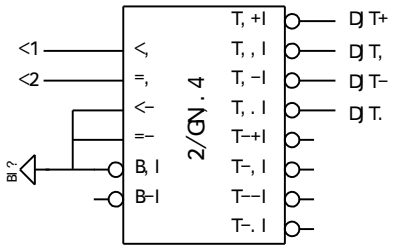


图 3-1　I/O 地址译码原理图

表 3-1　I/O 地址空间分配

A7　A6	选定	地址空间
00	IOY0	00-3F
01	IOY1	40-7F
10	IOY2	80-BF
11	IOY3	C0-FF

为了实现对于 MEM 和外设的读写操作，还需要一个读写控制逻辑，使得 CPU 能控制 MEM 和 I/O 设备的读写，实验中的读写控制逻辑如图 3-2 所示，由于 T3 的参与，可以保证写脉宽与 T3 一致，T3 由时序单元的 TS3 给出。IOM 用来选择是对 I/O 设备还是对 MEM 进行读写操作，IOM=1 时

对 I/O 设备进行读写操作，IOM=0 时对 MEM 进行读写操作。RD=1 时为读，WR=1 时为写。

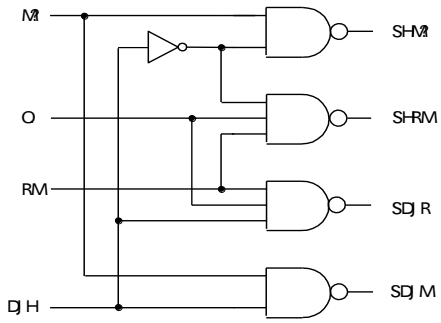


图 3-2　读写控制逻辑

在理解读写控制逻辑的基础上我们设计一个总线传输的实验。实验所用总线传输实验框图如图 3-3 所示，它将几种不同的设备挂至总线上，有存储器、输入设备、输出设备、寄存器。这些设备都需要有三态输出控制，按照传输要求恰当有序的控制它们，就可实现总线信息传输。

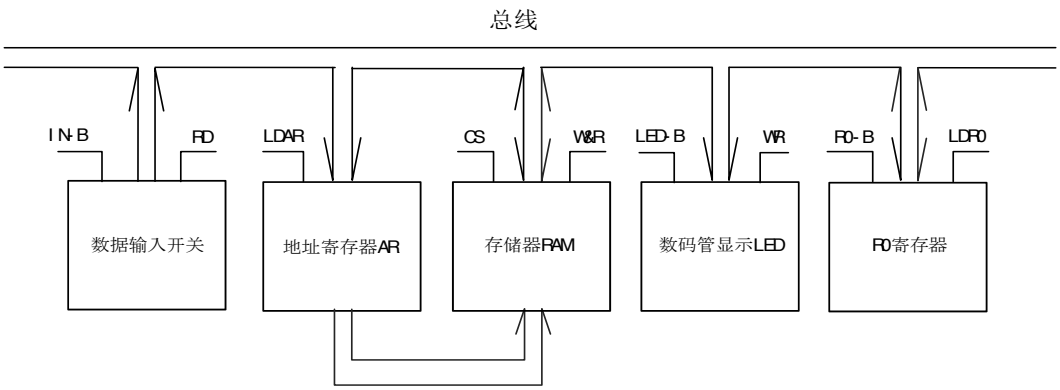


图 3-3　总线传输实验框图

四、实验步骤

基本输入输出功能的总线接口实验。

（1）根据挂在总线上的几个基本部件，设计一个简单的流程：

① 输入设备将一个数打入 R0 寄存器。

② 输入设备将另一个数打入地址寄存器。

③ 将 R0 寄存器中的数写入到当前地址的存储器中。

④ 将当前地址的存储器中的数用 LED 数码管显示。

（2）按照图 3-4 实验接线图进行连线。

（3）具体操作步骤图示如下：

进入软件界面，选择菜单命令“【实验】—【简单模型机】”，打开简单模型机实验数据通路图。

将时序与操作台单元的开关 KK1、KK3 置为‘运行’档，开关 KK2 置为‘单拍’档，CON 单元所有开关置 0（由于总线有总线竞争报警功能，在操作中应当先关闭应关闭的输出开关，再打开应打开的输出开关，否则可能由于总线竞争导致实验出错）， 按动 CON 单元的总清按钮 CLR，然后通过运行程序，在数据通路图中观测程序的执行过程。

① 输入设备将 11H 打入 R0 寄存器。

将 IN 单元置 00010001，K7 置为 1，关闭 R0 寄存器的输出；K6

置为 1，打开 R0 寄存器的输入；WR、RD、IOM 分别置为 0、1、1，对 IN 单元进行读操作；LDAR 置为 0，不将数据总线的数打入地址寄存器。连续四次点击图形界面上的“单节拍运行”按扭（运行一个机器周期），观察图形界面，在 T4 时刻完成对寄存器 R0 的写入操作。

② 将 R0 中的数据 11H 打入存储器 01H 单元。

将 IN 单元置 00000001（或其他数值）。K7 置为 1，关闭 R0 寄存器的输出；K6 置为 0，关闭 R0 寄存器的输入；WR、RD、IOM 分别置为 0、1、1，对 IN 单元进行读操作；LDAR 置为 1，将数据总线的数打入地址寄存器。连续四次点击图形界面上的“单节拍运行”按扭，观察图形界面，在 T3 时刻完成对地址寄存器的写入操作。

先将 WR、RD、IOM 分别置为 1、0、0，对存储器进行写操作；再把 K7 置为 0，打开 R0 寄存器的输出；K6 置为 0，关闭 R0 寄存器的输入； LDAR 置为 0，不将数据总线的数打入地址寄存器。连续四次点击图形界面上的“单节拍运行”按扭，观察图形界面，在 T3 时刻完成对存储器的写入操作。

③ 将当前地址的存储器中的数写入到 R0 寄存器中。

将 IN 单元置 00000001（或其他数值），K7 置为 1，关闭 R0 寄存器的输出；K6 置为 0，关闭 R0 寄存器的输入；WR、RD、IOM 分别置

为 0、1、1，对 IN 单元进行读操作；LDAR 置为 1，不将数据总线的数打入地址寄存器。连续四次点击图形界面上的“单节拍运行”按扭，观察图形界面，在 T3 时刻完成对地址寄存器的写入操作。

将 K7 置为 1，关闭 R0 寄存器的输出；K6 置为 1，打开 R0 寄存器的输入；WR、RD、IOM 分别置为 0、1、0，对存储器进行读操作；LDAR 置为 0，不将数据总线的数打入地址寄存器。连续四次点击图形界面上的“单节拍运行”按扭，观察图形界面，在 T3 时刻完成对寄存器 R0 的写入操作。

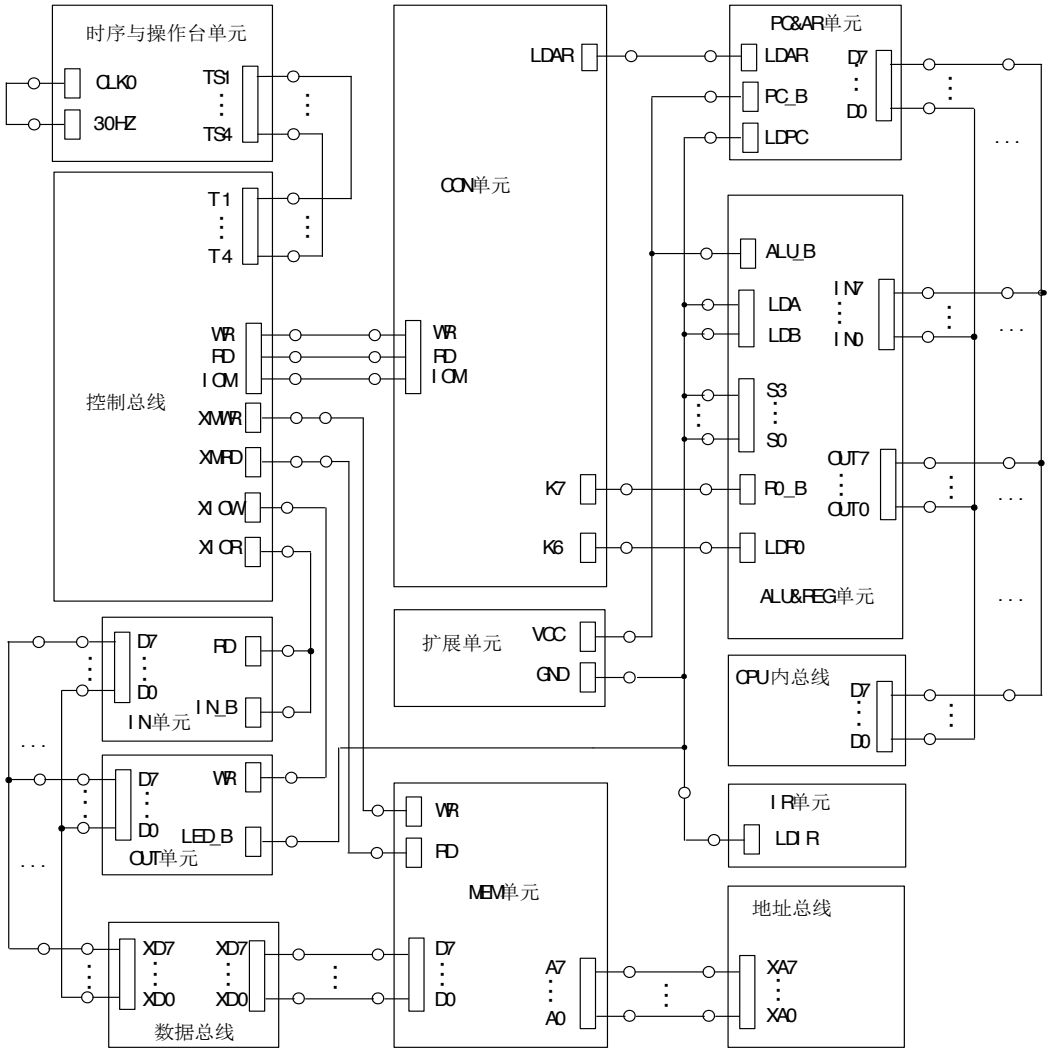
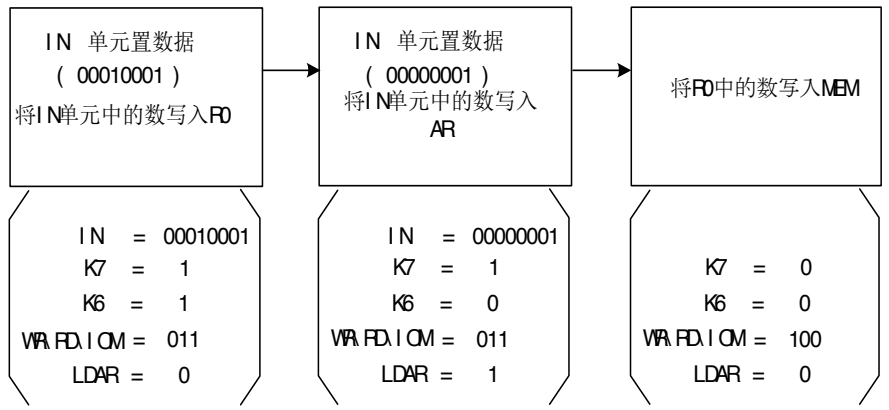


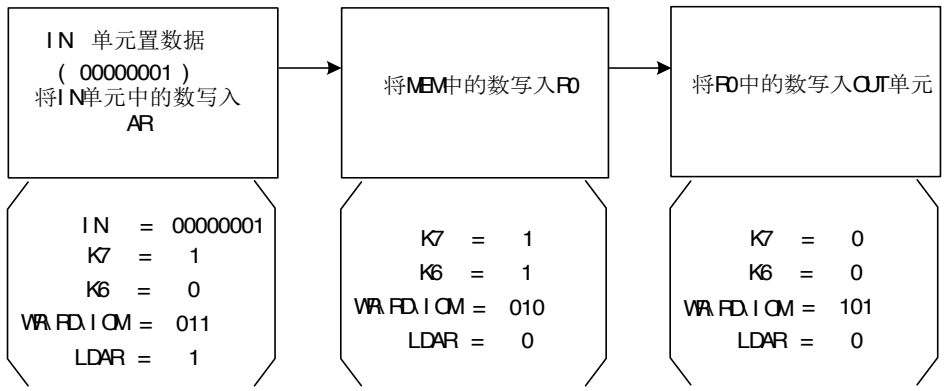
图 3-4　实验接线图

④　将 R0 寄存器中的数用 LED 数码管显示。

先将 WR、RD、IOM 分别置为 1、0、1，对 OUT 单元进行写操作；再将 K7 置为 0，打开 R0 寄存器的输出；K6 置为 0，关闭 R0 寄存器的输入； LDAR 置为 0，不将数据总线的数打入地址寄存器。连续四次点

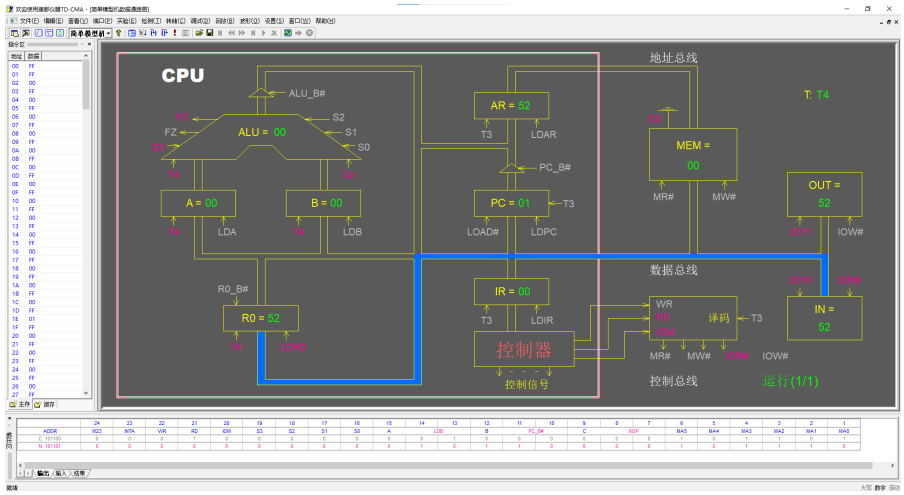
击图形界面上的“单节拍运行”按扭，观察图形界面，在 T3 时刻完成对 OUT 单元的写入操作。



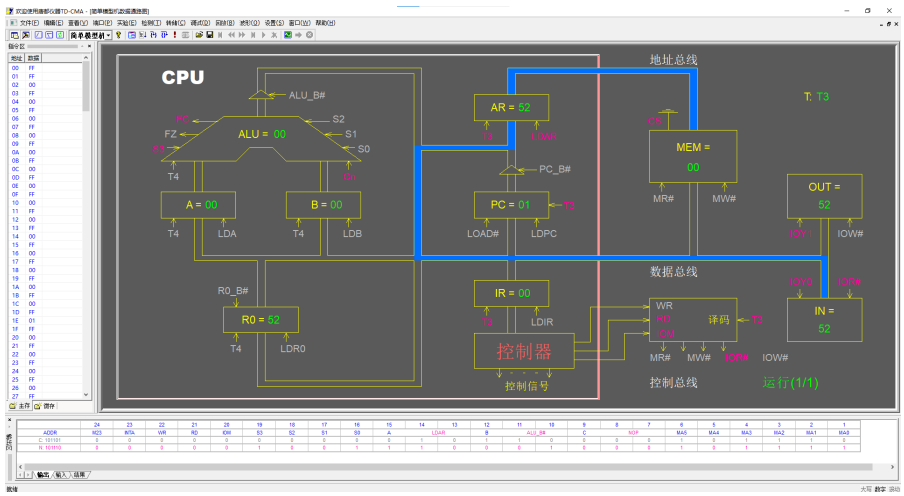


五、实验结果

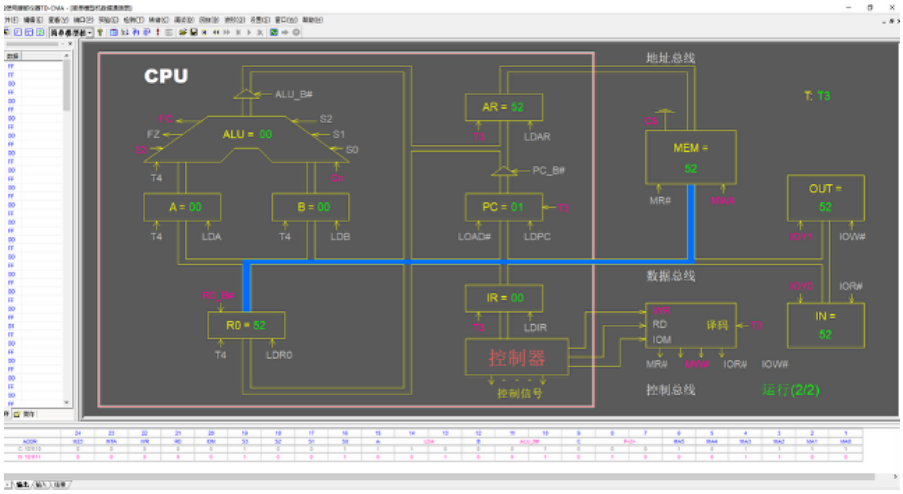
将学号尾号 96 写入 IN 单元并传入 R0:



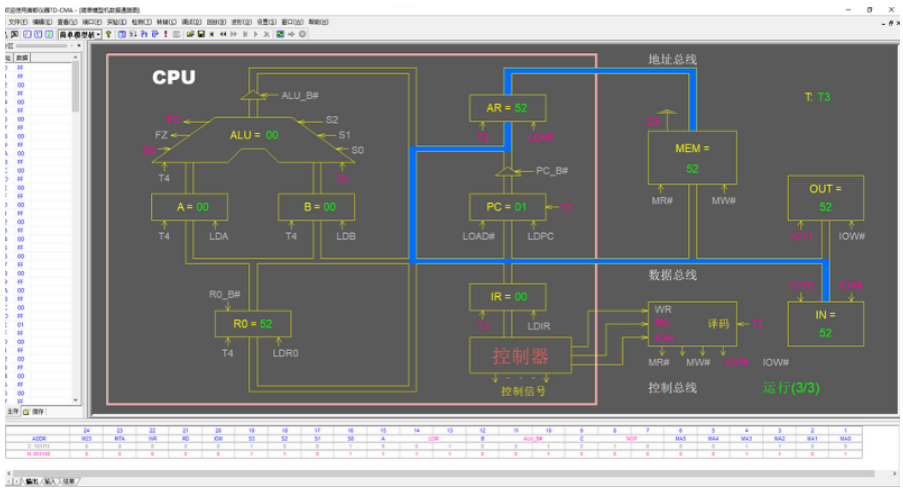
将 IN 单元内数据写入 AR；



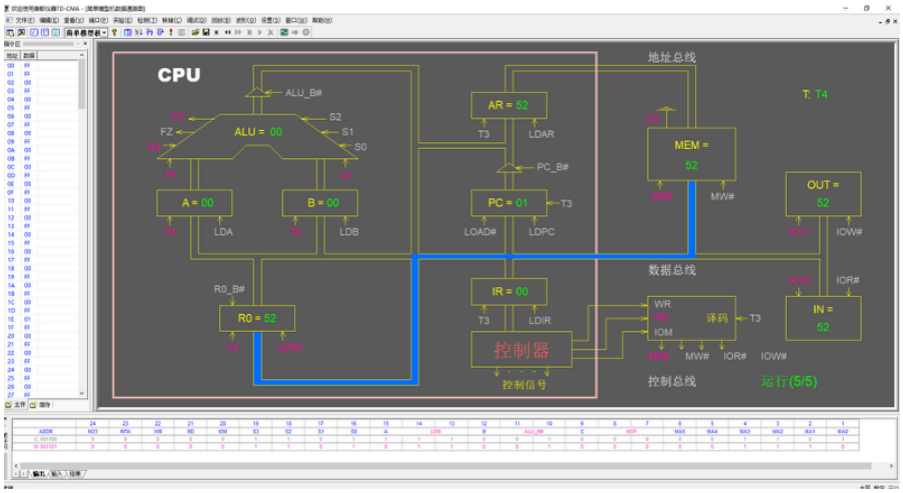
将 R0 内数据写入 MEM:



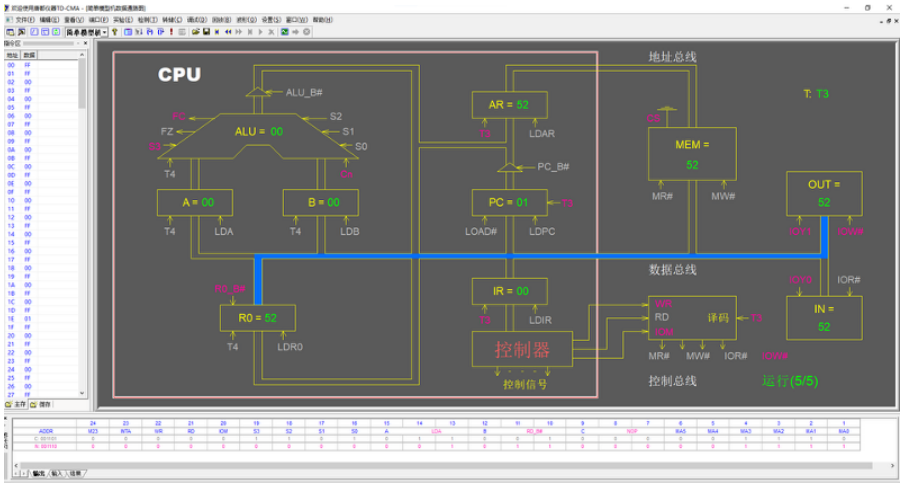
将 IN 单元内数据写入 AR：



将 MEM 内数据写入 R0：



将 R0 内数据写入 OUT 单元：



实验五　简单模型机设计实验

在前面的章节中，我们重点讨论计算机中每个部件的组成及特性，本实验中，我们将重点讨论如何完整设计一台模型计算机，进一步建立整机的概念。

一、实验目的

(1) 掌握一个简单 CPU 的组成原理。

(2) 在掌握部件单元电路的基础上，进一步将其构造一台基本模型计算机。

(3) 为其定义五条机器指令，编写相应的微程序，并上机调试掌握整机概念。

二、实验设备

PC 机一台，TD-CMA 实验系统一套。

三、实验原理

本实验要实现一个简单的 CPU，并且在此 CPU 的基础上，继续构建一个简单的模型计算机。CPU 由运算器（ALU)、微程序控制器（MC)、通用寄存器（R0)，指令寄存器（IR)、程序计数器（PC）和地址寄存器（AR）组成,如图 5-1 所示。这个 CPU 在写入相应的微指令后，就具备了执行机器指令的功能,但是机器指令一般存放在主存当中,CPU 必须和主存挂接后，

才有实际的意义，所以还需要在该 CPU 的基础上增加一个主存和基本的输入输出部件，以构成一个简单的模型计算机。

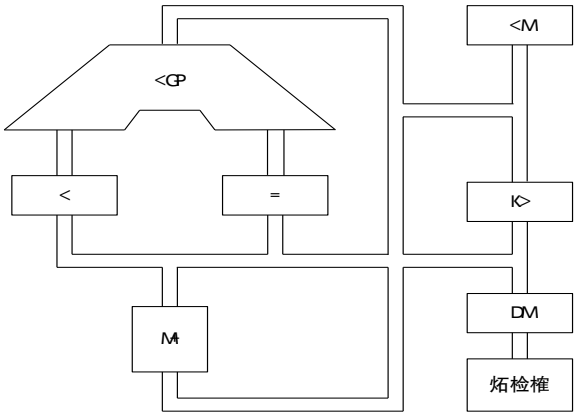


图 5-1　基本 CPU 构成原理图

除了程序计数器（PC），其余部件在前面的实验中都已用到，在此不再讨论。系统的程序计数器（PC）和地址寄存器（AR）集成在一片 CPLD 芯片中 。CLR 连接至 CON 单元的总清端 CLR，按下 CLR 按钮，将使 PC 清零，LDPC 和 T3 相与后作为计数器的计数时钟，当 LOAD 为低时，计数时钟到来后将 CPU 内总线上的数据打入 PC。

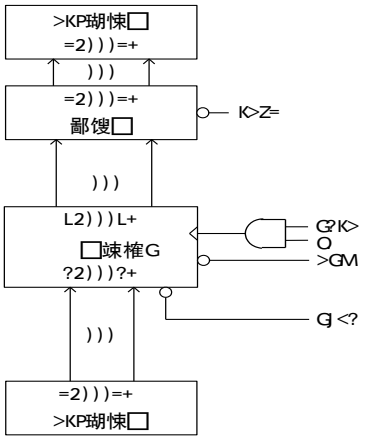


图 5-2　程序计数器(PC)原理图

本模型机和前面微程序控制器实验相比，新增加一条跳转指令 JMP，共有五条指令：IN（输入)、ADD（二进制加法)、OUT（输出)、JMP（无条件转移)，HLT（停机)，其指令格式如下（高 4 位为操作码)：

荆□□	雯榷贽宠□		□蝓
D	++, + ++++		D → M
<??	++++ ++++		M & M → M
J PO	++, , ++++		M → J PO
EHK __m	, , , + ++++	%%%%%%%	__m→ K>
CGO	+, +, ++++		疯雯

其中 JMP 为双字节指令，其余均为单字节指令，********为 addr 对应的二进制地址码。微程序控制器实验的指令是通过手动给出的，现在要求 CPU 自动从存储器读取指令并执行。根据以上要求，设计数据通路图，如图 5-3 所示。

本实验在前一个实验的基础上增加了三个部件，一是 PC(程序计数器)，另一个是 AR（地址寄存器)，还有就是 MEM（主存)。因而在微指令中应增加相应的控制位，其微指令格式如表 5-1 所示。

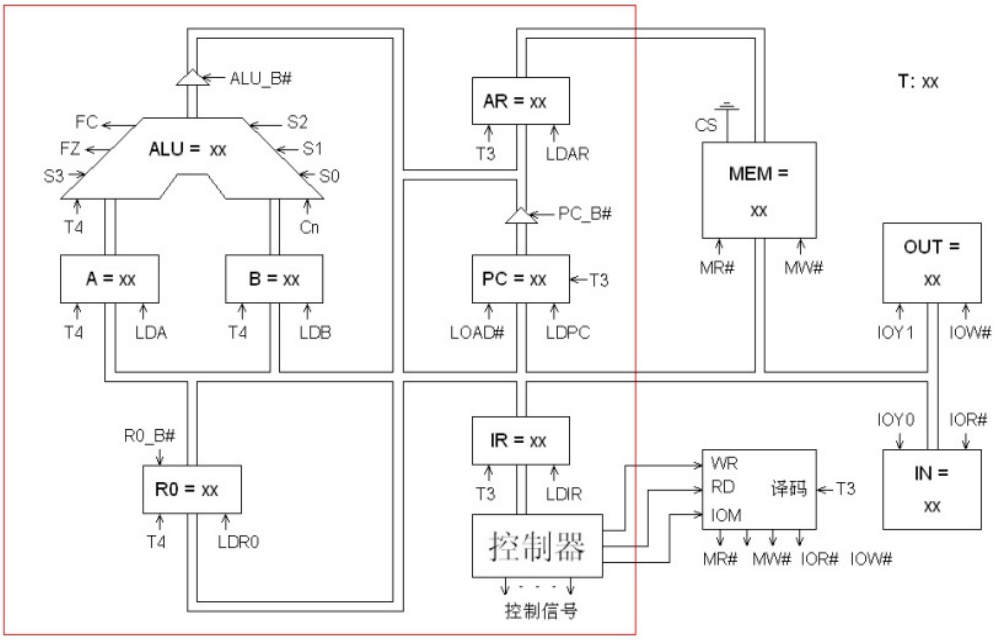


图 5-3　数据通路图

表 5-1 微指令格式

−.	−−	−,	−+	, 4	, 3(, 0	, /(, −	, , (4	3(1	0(+
H−.	H+−	RM	M?	DJ H	N (N+	<者□	=者□	>者□	H<0(H<+

<者□

, /	, .	, −	□昀
+	+	+	I J K
+	+	,	G?<
+	,	+	G?=
+	,	,	G?M<
,	+	+	肚□
,	+	,	GJ <?
,	,	+	G?<M
,	,	,	G?DM

=者□

, ,	, +	4	□昀
+	+	+	I J K
+	+	,	<GPZ=
+	,	+	M<Z=
+	,	,	肚□
,	+	+	肚□
,	+	,	肚□
,	,	+	K>Z=
,	,	,	肚□

>者□

3	2	1	□昀
+	+	+	I J K
+	+	,	K7, 9
+	,	+	肚□
+	,	,	肚□
,	+	+	肚□
,	+	,	G?K>
,	,	+	肚□
,	,	,	肚□

系统涉及到的微程序流程见图 5-4 所示，当拟定“取指”微指令时，该微指令的判别测试字段为 P<1>测试。指令译码原理见图 4-3 所示，由于“取指”微指令是所有微程序都使用的公用微指令，因此 P<1> 的测试结

果出现多路分支。本机用指令寄存器的高 6 位（IR7—IR2）作为测试条件，出现 5 路分支，占用 5 个固定微地址单元，剩下的其它地方就可以一条微指令占用控存一个微地址单元随意填写，微程序流程图上的单元地址为 16 进制。

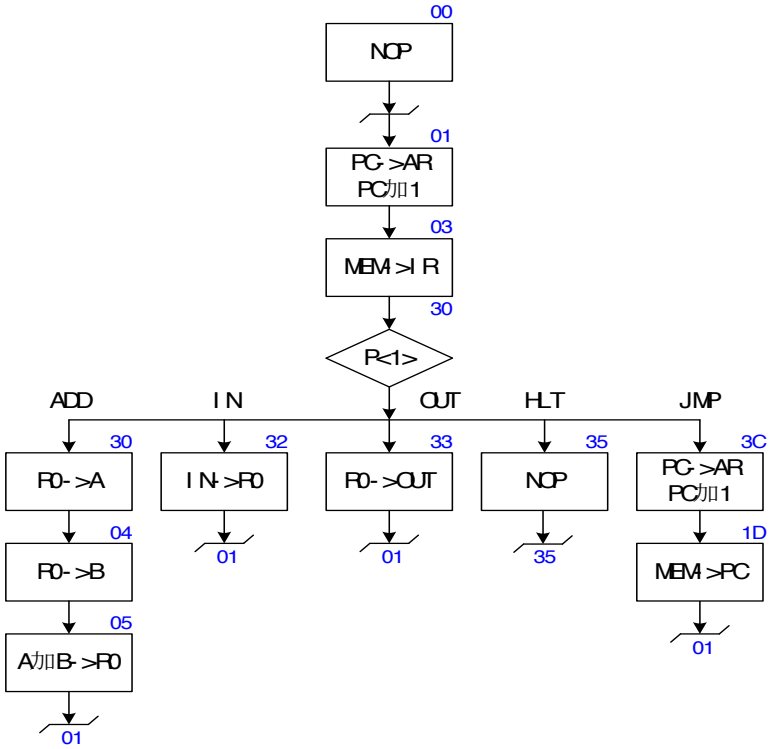


图 5-4　简单模型机微程序流程图

当全部微程序设计完毕后，应将每条微指令代码化，表 5-2 即为将图 5-4 的微程序流程图按微指令格式转化而成的“二进制微代码表”。

表 5-2　二进制微代码表

地址	十六进制	高五位	S3-S0	A 字段	B 字段	C 字段	MA5-MA0
00	00 00 01	00000	0000	000	000	000	000001

01	00 6D 43	00000	0000	110	110	101	000011
03	10 70 70	00010	0000	111	000	001	110000
04	00 24 05	00000	0000	010	010	000	000101
05	04 B2 01	00000	1001	011	001	000	000001
1D	10 51 41	00010	0000	101	000	101	000001
30	00 14 04	00000	0000	001	010	000	000100
32	18 30 01	00011	0000	011	000	000	000001
33	28 04 01	00101	0000	000	010	000	000001
35	00 00 35	00000	0000	000	000	000	110101
3C	00 6D 5D	00000	0000	110	110	101	011101

设计一段机器程序，要求从 IN 单元读入一个数据，存于 R0，将 R0 和自身相加，结果存于 R0，再将 R0 的值送 OUT 单元显示。

根据要求可以得到如下程序，地址和内容均为二进制数。

地 址	内 容	助记符	说 明
00000000	00100000	; START: IN R0	从 IN 单元读入数据送 R0
00000001	00000000	; ADD R0,R0	R0 和自身相加，结果送 R0
00000010	00110000	; OUT R0	R0 的值送 OUT 单元显示
00000011	11100000	; JMP START	跳转至 00H 地址
00000100	00000000	;	
00000101	01010000	; HLT	停机

四、实验步骤

1. 按图 5-5 连接实验线路。

2. 写入实验程序，并进行校验，分两种方式，手动写入和联机写入。

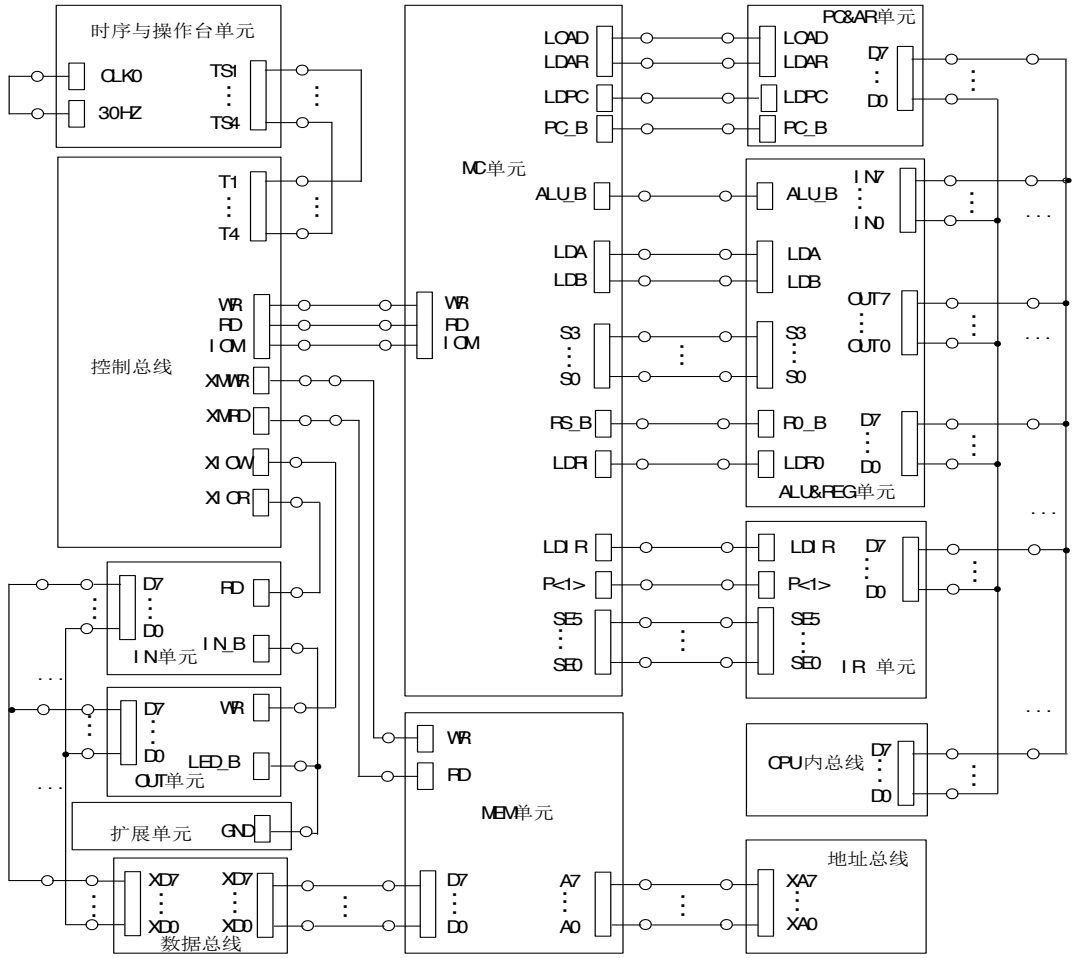
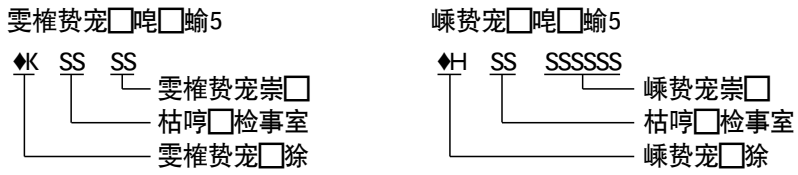


图 5-5　实验接线图

2) 联机写入和校验

联机软件提供了微程序和机器程序下载功能，以代替手动读写微程序和机器程序，但是微程序和机器程序得以指定的格式写入到以 TXT 为后缀

的文件中，微程序和机器程序的格式如下：



本次实验程序如下，程序中分号‘;’为注释符，分号后面的内容在下载时将被忽略掉:

```
; //*************************************//
; //                                     //
; //     CPU 与简单模型机实验指令文件    //
; //                                     //
; //     By TangDu CO.,LTD               //
; //                                     //
; //*************************************//
; //****** Start Of Main Memory Data ******//

  $P 00 20    ; START: IN  R0     从 IN 单元读入数据送 R0
  $P 01 00    ; ADD R0,R0         R0 和自身相加，结果送 R0
  $P 02 30    ; OUT R0            R0 的值送 OUT 单元显示
  $P 03 E0    ; JMP START         跳转至 00H 地址
  $P 04 00    ;
  $P 05 50    ; HLT               停机
; //******* End Of Main Memory Data *******//
; //**** Start Of MicroController Data ****//
  $M 00 000001    ; NOP
  $M 01 006D43    ; PC->AR,PC 加 1
  $M 03 107070    ; MEM->IR, P<1>
  $M 04 002405    ; R0->B
  $M 05 04B201    ; A 加 B->R0
  $M 1D 105141    ; MEM->PC
  $M 30 001404    ; R0->A
  $M 32 183001    ; IN->R0
  $M 33 280401    ; R0->OUT
  $M 35 000035    ; NOP
  $M 3C 006D5D    ; PC->AR,PC 加 1

; //** End Of MicroController Data **//
```

选择联机软件的“【转储】—【装载】”功能，在打开文件对话框中选择上面所保存的文件，软件自动将机器程序和微程序写入指定单元。

选择联机软件的“【转储】—【刷新指令区】”可以读出下位机所有的机器指令和微指令，并在指令区显示，对照文件检查微程序和机器程序是否正确，如果不正确，则说明写入操作失败，应重新写入，可以通过联机软件单独修改某个单元的指令，以修改微指令为例，先用鼠标左键单击指令区的‘微存’TAB 按钮，然后再单击需修改单元的数据，此时该单元变为编辑框，输入 6 位数据并回车，编辑框消失，并以红色显示写入的数据。

3. 运行程序

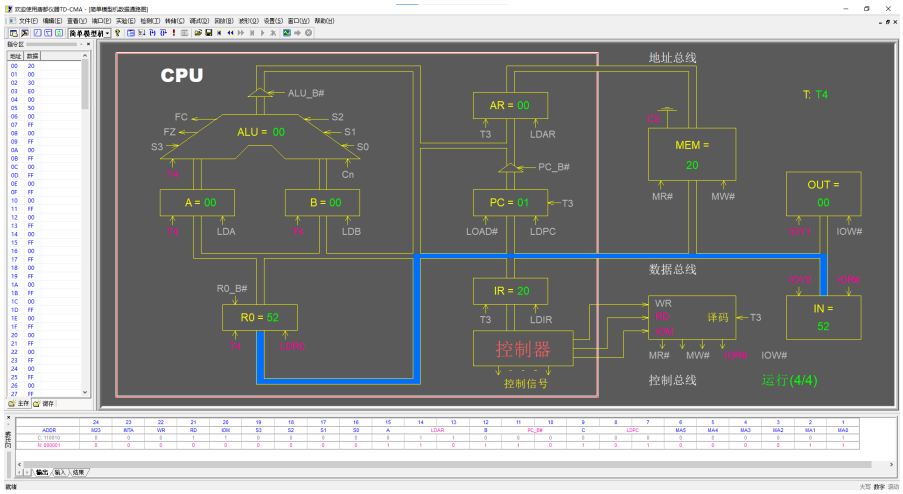
方法二：联机运行

将时序与操作台单元的开关 KK1 和 KK3 置为‘运行’档，进入软件界面，选择菜单命令“【实验】—【简单模型机】”，打开简单模型机数据通路图。

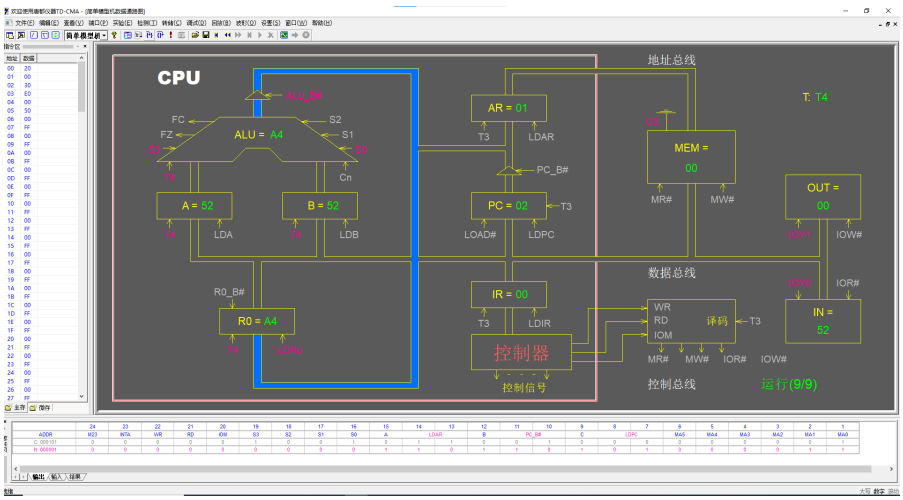
按动 CON 单元的总清按钮 CLR，然后通过软件运行程序，选择相应的功能命令，即可联机运行、监控、调试程序，当模型机执行完 JMP 指令后，检查 OUT 单元显示的数是否为 IN 单元值的 2 倍。在数据通路图和微程序流中观测指令的执行过程，并观测软件中地址总线、数据总线以及微指令显示和下位机是否一致。

五、实验结果

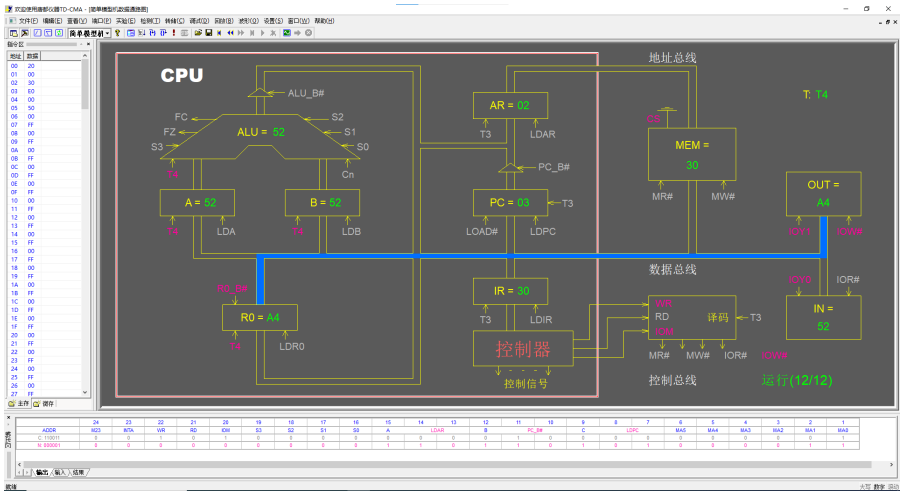
将学号尾号 96 写入 IN 单元并写入 R0:



R0 内数据送至 ALU 的 A 端：



R0 内数据送至 ALU 的 B 端;



R0 内数据写入 OUT 单元:

